

Bases de datos relacionales y SQL básico

De preguntarle a un DataFrame a preguntarle a una base de datos

Módulo: Módulo 1: Explorar (Fundamentos y Consulta de Datos)

Semana 3 · Clase 5 · Sep 22 al 27 · Virtual

Docente: Cristian A. Jimenez C., MSc — Sep 22 al 27

Hora 1	Hora 2	Hora 3
Bases de datos relacionales: tabla, registro, campo, clave primaria y esquema relacional	SQL básico: SELECT, FROM, WHERE, ORDER BY, LIMIT y operadores	Laboratorio: consultas de extracción y filtrado sobre una base de datos empresarial

En la Clase 4 le hicimos preguntas de negocio a un DataFrame de pandas (filtrar, contar, resumir). Hoy volvemos a hacer las mismas preguntas, pero sobre datos que viven en una **base de datos relacional**, con un lenguaje hecho exactamente para eso: **SQL**.

Una **base de datos** es un conjunto de datos organizados para guardarse, consultarse y actualizarse de forma confiable, generalmente gestionado por un programa especializado (un **motor de base de datos** o *DBMS*).

¿Y por qué no simplemente un Excel gigante?

- ▶ Varias personas (o programas) pueden leer y escribir **al mismo tiempo**, sin sobrescribirse.
- ▶ Los datos se organizan en **varias tablas relacionadas** en vez de una sola hoja enorme y repetitiva.
- ▶ El motor puede **exigir reglas**: que un campo no quede vacío, que un identificador no se repita, que una referencia exista.
- ▶ Consultar millones de filas es mucho más rápido que en una hoja de cálculo.

Hoy usamos **SQLite**: un motor de base de datos liviano que vive en un solo archivo (.db), ideal para aprender sin instalar un servidor.

Una base de datos **relacional** organiza la información en **tablas**, cada una parecida a una hoja de cálculo con reglas más estrictas.

Tabla = clientes

id_cliente	nombre	ciudad	segmento
1	Julián Ramírez Restrepo	Bucaramanga	Empresa
2	Valentina Álvarez Castaño	Medellín	Estudiante
3	Camila Jaramillo Castaño	Cali	Hogar

Campo (columna)
un atributo de cada cliente

Registro (fila)
un cliente completo

Clave primaria
identifica cada registro,
sin repetirse

Clave primaria (PK): el campo que identifica **sin ambigüedad** cada registro de una tabla; no se repite y no puede quedar vacío. Ejemplo: `id_cliente` en la tabla `clientes`.

¿Por qué no usar el nombre como identificador?

Porque los nombres **se repiten** (dos clientes pueden llamarse igual) y **pueden cambiar** (un cliente puede corregir cómo está escrito su nombre). Un identificador numérico no tiene ninguno de esos problemas.

Clave foránea (FK): un campo que **apunta a la clave primaria de otra tabla**, y así conecta las dos tablas. Ejemplo: en `pedidos`, el campo `id_cliente` reconoce *a qué cliente pertenece* cada pedido.

Hoy trabajamos con `tienda.db`: una tienda en línea ficticia (“Tienda Andina”), con 5 tablas relacionadas entre sí mediante claves primarias y foráneas.



Porque **en el trabajo real casi nunca eliges tú dónde viven los datos**: muchas veces están en una base de datos, y para traerlos a Python (o a Power BI, Módulo 3) primero hay que saber pedírselos con SQL.

Lo que ya sabes de la Clase 4, con otro nombre

- ▶ Filtrar filas de un DataFrame → cláusula WHERE.
- ▶ Elegir columnas → lo que escribes después de SELECT.
- ▶ `.sort_values()` → ORDER BY.

SQL (*Structured Query Language*) es el lenguaje estándar para hacerle preguntas a una base de datos relacional: se aprende una vez y sirve para (casi) cualquier motor (SQLite, PostgreSQL, MySQL, SQL Server...).

Toda consulta parte de la misma idea: de una tabla completa, pedimos **ciertas columnas** de **ciertas filas**.

```
SELECT nombre, ciudad  
FROM clientes;
```

- ▶ **SELECT**: **qué columnas** quiero ver.
- ▶ **FROM**: **de qué tabla** las quiero.
- ▶ Toda instrucción SQL termina en **punto y coma (;)**.

El resultado de una consulta es siempre otra tabla

Así tenga una sola columna o una sola fila: el resultado de un **SELECT** se ve y se comporta como una tabla nueva.

Columnas específicas (lo recomendado casi siempre):

```
SELECT nombre, precio  
FROM productos;
```

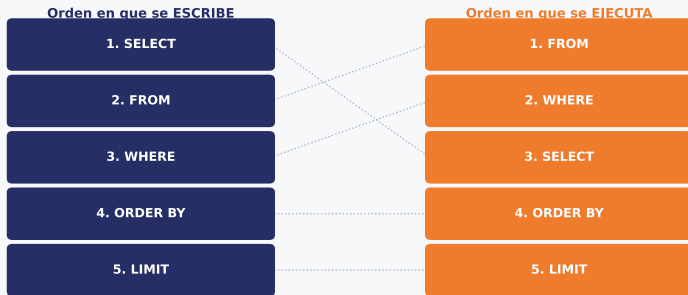
Todas las columnas, con *:

```
SELECT *  
FROM productos;
```

¿Por qué no usar siempre *?

Trae columnas que quizás no necesitas (más lento y más desordenado de leer), y si alguien le agrega una columna a la tabla mañana, tu consulta con * cambia de resultado sin que tú lo hayas pedido. Nombrar las columnas es más explícito y más seguro.

Un detalle que suele confundir al principio: SQL **no** ejecuta la consulta en el mismo orden en que la escribes.



El motor arma primero la tabla (FROM), filtra filas (WHERE) y solo al final decide qué columnas mostrar (SELECT).

Por eso **WHERE** puede filtrar por una columna que ni siquiera aparece en el **SELECT**: el motor ya armó la tabla completa (**FROM**) y la filtró (**WHERE**) **antes** de decidir qué mostrar.

WHERE filtra filas: solo pasan las que cumplen la condición (exactamente como `df[condición]` en pandas).

Operador	Significado
=	igual a
!= ó <>	distinto de
> / <	mayor que / menor que
>= / <=	mayor o igual / menor o igual

```
SELECT nombre, ciudad
FROM clientes
WHERE ciudad = 'Medellín';
```

Los valores de texto van entre **comillas simples** ('Medellín'); los números, sin comillas (precio > 200000).

Sobre la tabla productos de tienda.db:

```
SELECT nombre, precio
FROM productos
WHERE categoria = 'Tecnología' AND precio > 200000;
```

nombre	precio
Portátil 14 pulgadas	2 450 000
Monitor 24 pulgadas	780 000
Teclado mecánico	245 000

Dos condiciones unidas con AND: solo pasan las filas que cumplen **ambas** a la vez.

- ▶ AND: deben cumplirse **todas** las condiciones.
- ▶ OR: basta con que se cumpla **alguna**.
- ▶ NOT: invierte una condición.

Cuidado con mezclar AND y OR

SQL evalúa AND **antes** que OR (igual que la multiplicación antes que la suma). Si no es lo que quieres, usa **paréntesis** para dejarlo explícito.

Sin paréntesis (ambiguo):

```
WHERE categoria = 'Hogar'  
      OR categoria = 'Oficina'  
      AND precio > 500000;
```

Con paréntesis (lo que sí queríamos):

```
WHERE (categoria = 'Hogar'  
      OR categoria = 'Oficina')  
      AND precio > 500000;
```

La primera versión trae **todo** Hogar (sin importar el precio) más Oficina caro; la segunda solo trae Hogar u Oficina, ambos por encima de \$500.000. Son dos resultados distintos con

Cuatro atajos muy usados para condiciones comunes:

IN: pertenece a una lista

```
WHERE ciudad IN  
('Cali', 'Bogotá');
```

BETWEEN: dentro de un rango

```
WHERE fecha BETWEEN  
'2026-03-01' AND  
'2026-03-31';
```

LIKE: coincide con un patrón (% = cualquier texto):

```
WHERE nombre LIKE  
'%portátil%';
```

IS NULL: el campo está vacío

```
WHERE correo IS NULL;
```

¿Por qué no `correo = NULL`?

Porque NULL significa “valor desconocido”, y nada es *igual* a algo desconocido, ni siquiera otro NULL. Por eso existe el operador especial `IS NULL` (y `IS NOT NULL`).

ORDER BY ordena las filas del resultado por una o más columnas. Por defecto es ascendente (ASC); para descendente se agrega DESC.

```
SELECT nombre, precio
FROM productos
ORDER BY precio DESC;
```

nombre	precio
Portátil 14 pulgadas	2 450 000
Escritorio ajustable	1 150 000
Monitor 24 pulgadas	780 000
...	...

LIMIT corta el resultado a las primeras n filas, muy útil para explorar una tabla grande sin traerla completa, o para preguntas tipo “los 3 productos más caros”.

```
SELECT nombre, precio
FROM productos
ORDER BY precio DESC
LIMIT 3;
```

nombre	precio
Portátil 14 pulgadas	2 450 000
Escritorio ajustable	1 150 000
Monitor 24 pulgadas	780 000

Nota el orden real de ejecución (diapositiva anterior): primero se arma y filtra la tabla, **luego** se ordena, y **al final** se recorta con LIMIT.

Pregunta de negocio: “¿cuáles son los 3 productos con menos stock disponible?”

```
SELECT nombre, stock
FROM productos
WHERE stock < 15
ORDER BY stock ASC
LIMIT 3;
```

nombre	stock
Escritorio ajustable	8
Archivador metálico	10
Silla ergonómica	12

Filtrar, ordenar y limitar se combinan todo el tiempo: la estructura siempre es SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT.

Reto de la sesión

Practicar consultas de extracción y filtrado sobre `tienda.db`, una base de datos SQLite de una tienda en línea ficticia, con las mismas 5 tablas del esquema relacional que vimos hoy.

Dos caminos posibles, con la misma base de datos (`datos/tienda.db` en el repositorio del curso):

- ▶ **Google Colab** (como en la Clase 4): subes `tienda.db` con `files.upload()` y consultas con Python + `sqlite3`: cuaderno guiado paso a paso.
- ▶ **DBeaver** (opcional, si quieres una herramienta visual de base de datos): te conectas directamente al mismo archivo `tienda.db`, sin instalar ningún servidor.

No hace falta ningún servidor en la nube: SQLite vive en un solo archivo, y ese archivo es el que descargas y usas en cualquiera de los dos caminos.

Entregable

Completar el cuaderno `lab-sql-basico.ipynb`: las consultas guiadas de cada sección, las actividades marcadas con `-- TODO`, y el reto final de 5 preguntas de negocio sobre `tienda.db` (equivalente en SQL al reto de la Clase 4 en pandas).

Guarda una copia del cuaderno en tu Google Drive antes de cerrar Colab (el archivo `tienda.db` que subiste es efímero, igual que vimos en la Clase 4).

- ▶ Una base de datos relacional organiza la información en **tablas** conectadas por **claves primarias y foráneas**.
- ▶ SELECT, FROM, WHERE, ORDER BY y LIMIT responden, con otro lenguaje, las mismas preguntas de negocio que ya resolvías con pandas en la Clase 4.
- ▶ Próxima sesión, **Clase 6, MasterClass**: recapitulación presencial de todo el Módulo 1 (estadística descriptiva, Python y SQL) antes de entrar al Módulo 2 (Pandas a fondo).
- ▶ En la **Clase 7** retomamos pandas, ahora para diagnóstico de calidad de datos: nulos, duplicados y tipos; muchas de las condiciones que hoy escribiste en WHERE (como IS NULL) reaparecen ahí.

¡Gracias!